हेलो! आपका स्वागत है **Unit 11: this, Objects & Prototypes** की complete study class में.

मैंने आपके workspace में, unit 11 this prototype folder के अंदर ये complete study files write कर दी हैं:

- Study Guide File: chapter11_this_prototypes.md
- Code Examples File: this_prototypes_examples.js
- Practice Exercises File: this_prototypes_practice.js

चलिए अब OOPs, this behavior, prototype templates, inheritance, dynamic classes keys, methods patterns parameters ko simple Hinglish explanation ke sath samajhna shuru karte hain.

PART 1: THE this KEYWORD & CONSTRUCTORS

JavaScript functions dynamically contexts resolve logic variables refer bind rules run dynamic actions perform context values tracks.

1. the this Keyword Behavior

What is it? this keyword execution context binding checks hold variables refer object pointers points runtime calls.

How does it behave?

- **Global Scope Context:** points browser Window object (window) / NodeJS global pointer.
- **Method call context:** points owner object (profile.showName() → owner is profile).
- **Regular function call context:** non-strict mode defaults to global object; strict mode maps to undefined.
- **Arrow functions:** Arrow functions do not declare personal this pointers. They lexically resolve value by looking up parent block surrounding environment.

Simple example

javascript

```
const userProfile = {
  username: "Ravi",
  printName: function() {
    console.log(this.username); // 'this' points to userProfile
```

```
    }
};
userProfile.printName();
```

Output Ravi

---

2. Constructor Functions & new Keyword

What is it?

- **Constructor Function:** Blueprint function sets capital letter starts designed templates setup.
- **new operator:** Creates new object instance mapping templates definitions.

What new does step-by-step:

1. Creates new empty block object {}.
2. Prototype link setup assigns to constructor template prototype.
3. Binds execution context this to new empty object.
4. Executes constructor codes blocks and yields constructed object.

Simple example

javascript

```
function Player(name, team) {
    this.name = name;
    this.team = team;
}
let cricketer = new Player("Virat", "India");
console.log(cricketer.name);
```

Output Virat

PART 2: PROTOTYPES & PROTOTYPE CHAIN

1. Prototypes & Prototype Chain

What is it?

- **Prototype:** Link templates template parent reference configurations sharing capabilities.
- **Prototype Chain:** Look up reference check links: Local object parameters query match → Prototype mapping checks → base Object properties checking → null.

Simple example

javascript

let parent = { state: "Delhi" };

let child = Object.create(parent); // sets parent as prototype of child

console.log(child.state); // resolved from prototype chain link parent

Output Delhi

---

2. __proto__ Concept

What is it? Internal property reference matching link pointing to immediate prototype parent object.

Simple example

javascript

let animal = { eat: true };

let dog = {};

dog.__proto__ = animal; // links prototype parent

console.log(dog.eat);

Output true

PART 3: CLASSES & INHERITANCE

1. JavaScript Classes

What is it? Modern syntactic sugar wrapper classes layout structures ES6 specification (under the hood compiles prototypes).

Syntax

javascript

class UserAccount {

```
  constructor(email) {
    this.email = email;
  }
  showEmail() {
    return this.email;
  }
}
let activeUser = new UserAccount("ravi@mail.com");
console.log(activeUser.showEmail());
```

Output ravi@mail.com

---

2. Inheritance (extends & super)

What is it?

- **extends:** creates child sub class relationships templates.
- **super:** calls variables constructor properties parent classes initializations parameters.

Simple example

javascript

```
class Employee {
  constructor(role) {
    this.role = role;
  }
}
class Manager extends Employee {
  constructor(role, dept) {
    super(role); // calls Employee constructor
    this.dept = dept;
  }
}
```

let m = new Manager("Lead", "Engineering");

console.log(m.role);

Output Lead

PART 4: INTERVIEW SPECIALS

1. instanceof

What is it? Evaluates if an object exists within a constructor prototype chain.

Example

javascript

```
class Car {}

let sedan = new Car();

console.log(sedan instanceof Car); // true
```

---

2. Static Methods & Properties

What is it? Static declarations limit functions actions inside Class name directly. Individual instances cannot access static variables.

Example

javascript

```
class MathTool {
   static getPI() {
      return 3.14;
   }
}

console.log(MathTool.getPI());
```

Output 3.14

🧠 **QUICK REVISION SUMMARY**

- **this keyword:** Points to active execution owner context object. Lexical lookup in arrow functions.
- **new keyword:** Steps: Creates {} → Sets Proto link → Binds this to new object → Returns new object.
- **Prototypes:** Shared template properties links linking chain loops.
- **Classes:** sugar templates. extends creates subclasses, super forwards parent arguments parameters.
- **Static:** class level calls, hidden from constructor instances templates.

📝 IMPORTANT DEFINITIONS (INTERVIEW-READY)

1. **Prototype Chain:** The sequential lookup path JavaScript traverses to resolve object properties.
2. **Dunder Proto (__proto__):** The internal instance property link matching object instance to its direct parent prototype template.
3. **Constructor Blueprint:** Template functions generating identical object structures through the new keyword.
4. **Static Scope Bindings:** Standard methods bound to class definitions rather than class instances.
5. **Call Context Binding:** The dynamic execution context mapping target object references to this.

💬 IMPORTANT INTERVIEW QUESTIONS & ANSWERS

Q1. What does new keyword do internally?

Ans:

1. Creates empty object {}.
2. Link parent prototype sets to constructor prototype property.
3. Binds this to new empty object.
4. Runs constructor body code and returns the object.

Q2. Explain prototypes in JavaScript.

Ans: JavaScript objects have prototype properties template objects from which they inherit methods and properties.

Q3. Predict output:

javascript

```
const user = {
   name: "Ravi",
   greet: () => {
      console.log(this.name);
   }
};
user.greet();
```

Ans: undefined. Arrow functions inherit this lexically from their parent scope (global window context here), where name is undefined.

Q4. Difference between prototype and __proto__?

Ans: prototype is present on constructor functions defining the blueprint. __proto__ is the instance pointer pointing to the parent prototype object.

Q5. What is Prototype Chain?

Ans: The lookup chain JavaScript uses to search for properties, starting from local fields up to Object.prototype (pointing to null).

Q6. Difference between Constructor Function and Class?

Ans: Classes are syntactic sugar over prototypes. Invoking a class without new throws a TypeError.

Q7. What is super() used for?

Ans: To invoke the parent class constructor within the subclass, ensuring inherited properties are initialized correctly.

Q8. Predict output:

javascript

```
class Test {
   static show() {
      return "static";
   }
```

}

let t = new Test();

console.log(t.show);

Ans: undefined. Static methods belong to the class template, not to individual instances.

Q9. Explain instanceof check.

Ans: Evaluates whether a constructor's prototype exists within the prototype chain of an object.

Q10. What is a static method in a class?

Ans: Methods defined on the class itself, called directly on the class and not through instances.

Q11. Predict output:

javascript

function User() {}

let u = User();

console.log(u);

Ans: undefined. Since the function was called without new and lacks an explicit return statement.

Q12. Predict output:

javascript

let animal = { eat: true };

let dog = Object.create(animal);

console.log(dog.eat);

Ans: true. Property eat is inherited from the parent prototype.

Q13. What is the top of the prototype chain?

Ans: Object.prototype, which eventually points to null.

Q14. Predict output:

javascript

class Parent {}

class Child extends Parent {}

```
let c = new Child();

console.log(c instanceof Parent);
```

Ans: true. Since Child inherits from Parent, the base class is in its prototype chain.

Q15. Predict output:

javascript

```
function Test() {
   this.val = 10;
}

let t = new Test();

console.log(t.val);
```

Ans: 10.

Q16. Can arrow functions be used as constructors?

Ans: No. Arrow functions lack a prototype property and do not bind their own this context, so calling them with new throws a TypeError.

Q17. Predict output:

javascript

```
console.log(this); // running inside NodeJS file
```

Ans: module.exports (empty object {}), which is the default global module context in Node.js.

Q18. Predict output:

javascript

```
function show() {
   console.log(this);
}

show(); // running in non-strict browser mode
```

Ans: The window object.

Q19. Predict output of above function in strict mode:

Ans: undefined.

Q20. What happens if child class constructor misses calling super()?

Ans: It throws a ReferenceError. The child class constructor must call super() before accessing this.

💻 PRACTICE QUESTIONS

Question 1: Inheritance Setup Classes

- **Question:** Build a parent class Vehicle (brand) and child Car (brand, model).
- **Solution:**

javascript

```
class Vehicle {
  constructor(brand) {
    this.brand = brand;
  }
}
class Car extends Vehicle {
  constructor(brand, model) {
    super(brand);
    this.model = model;
  }
  showInfo() {
    return `${this.brand} ${this.model}`;
  }
}
let myCar = new Car("Ford", "Mustang");
console.log(myCar.showInfo()); // Ford Mustang
```

- **Explanation:** Inherits parent properties using extends and forwards them using super().

Question 2: Prototype property sharing methods

- **Question:** Attach a method greet to a constructor function prototype.

- **Solution:**

javascript

```
function User(name) {
   this.name = name;
}
User.prototype.greet = function() {
   return "Hello " + this.name;
};
let user1 = new User("Rahul");
console.log(user1.greet()); // Hello Rahul
```

- **Explanation:** Attaching the method to the prototype sharing it across all instances, saving memory.

Question 3: Instance check validation instanceof

- **Question:** Create prototype objects and verify relationships using instanceof.
- **Solution:**

javascript

```
class Human {}
let person = new Human();
console.log("Is person Human?", person instanceof Human); // true
```

- **Explanation:** Checks if the constructor's prototype exists in the object's prototype chain.